PROJECT PROBLEM STATEMENT | AGENTIC AI & RAG CAPSTONE TRACK

Corporate Learning Path Recommender with Context-Aware Agent

Personalized course-path planning agent combining catalog RAG, LMS tool calls, and multi-turn refinement

BUSINESS SCENARIO

A corporate training provider offers hundreds of courses across SAP, Workday, Salesforce, Cloud, AI, and Cybersecurity tracks. Learners frequently do not know which sequence of courses fits their career goals, current skill level, and completed work, resulting in poor course selection, wasted time, and low completion rates. Advisors cannot manually plan a path for every learner at the scale the program operates at, and existing course-search tools only support keyword lookup, not goal-based planning.

EXPECTED SOLUTION

A LangGraph agent that holds a short conversation with the learner about goals, background, and time availability, retrieves relevant course descriptions and prerequisite maps via RAG, and checks the learner's enrollment and completion history through a tool call to the LMS database. It composes a sequenced learning path with estimated hours and refines it as the learner adds constraints mid-conversation, for example stating they have no prior SAP experience or only 5 hours a week available.

KEY CHALLENGES TO SOLVE

- Balancing prerequisite ordering against a learner's stated time constraints without producing an unrealistic plan.
- Correctly revising an already-proposed path when new information contradicts an earlier assumption.
- Avoiding recommending courses the learner has already completed or is currently enrolled in.

EVALUATION RUBRIC

#	CRITERION	WEIGHT	WHAT IS MEASURED
1	**Recommendation Relevance**	40%	Proposed path matches stated goals and skill level, scored against expert-defined reference paths.
2	**Context-Aware Refinement**	35%	Agent updates the path correctly when new constraints are given mid-session, without contradicting earlier turns.
3	**RAG Retrieval Quality**	25%	RAGAS answer relevancy and context recall for retrieved course and prerequisite content.

EXPECTED WORKFLOW

1. Intake node collects goals, background, and time availability SEQ
2. RAG node retrieves course catalog and prerequisite content PAR tool node checks LMS enrollment/progress
3. Path composition node merges both outputs into a sequenced, time-boxed plan
4. Feedback loop: learner adds a constraint → re-enter composition node with updated context, prior plan retained as reference
5. Final path with estimated hours per course returned to the frontend

EXPECTED STACK

API: FastAPI

Frontend: React (or HTML) chat interface with a visual path/timeline view

Database: SQL database for course catalog, prerequisites, and learner enrollment records

RAG: Vector store over course descriptions and prerequisite maps, evaluated with RAGAS

Agent: LangGraph, sequential flow with a refinement loop-back

Tools: LMS progress and enrollment lookup API

Context: Goal-state tracking across turns so refinements do not lose earlier constraints